Stat 405 project

Jack

2026-04-07

Data Cleaning

```
library(cmdstanr)
```

```
## This is cmdstanr version 0.9.0
```

```
## - CmdStanR documentation and vignettes: mc-stan.org/cmdstanr
```

```
## - CmdStan path: /Users/jackyip/.cmdstan/cmdstan-2.38.0
```

```
## - CmdStan version: 2.38.0
```

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(ggplot2)
library(posterior)
```

```
## This is posterior version 1.7.0
```

```
##
## Attaching package: 'posterior'
```

```
## The following objects are masked from 'package:stats':
##
##     mad, sd, var
```

```
## The following objects are masked from 'package:base':
##
##     %in%, match
```

```
library(bayesplot)
```

```
## This is bayesplot version 1.15.0
```

```
## - Online documentation and vignettes at mc-stan.org/bayesplot
```

```
## - bayesplot theme set to bayesplot::theme_default()
```

```
##    * Does _not_ affect other ggplot2 plots
```

```
##    * See ?bayesplot_theme_set for details on theme setting
```

```
##
## Attaching package: 'bayesplot'
```

```
## The following object is masked from 'package:posterior':
##
##     rhat
```

```
library(coda)
```

```
## Warning: package 'coda' was built under R version 4.3.3
```

```
library(loo)
```

```
## Warning: package 'loo' was built under R version 4.3.3
```

```
## This is loo version 2.8.0
```

```
## - Online documentation and vignettes at mc-stan.org/loo
```

```
## - As of v2.0.0 loo defaults to 1 core but we recommend using as many as possible. Use
the 'cores' argument or set options(mc.cores = NUM_CORES) for an entire session.
```

```
##
## Attaching package: 'loo'
```

```
## The following object is masked from 'package:posterior':
##
##     gpdfit
```

```
library(tidyr)

housing_data <- read.csv("~/Downloads/melb_data.csv")
part1_data <- housing_data %>%
  mutate(
    log_price = log(Price),
    distance_z = as.numeric(scale(Distance)),
    buildingarea_z = as.numeric(scale(BuildingArea))
  ) %>%
  filter(
    !is.na(log_price),
    !is.na(distance_z),
    !is.na(buildingarea_z),
    !is.na(Regionname)
  )

part1_data$region_id <- as.integer(as.factor(part1_data$Regionname))

stan_data_model1 <- list(
  N = nrow(part1_data),
  y = part1_data$log_price,
  distance = part1_data$distance_z,
  buildingarea = part1_data$buildingarea_z
)

stan_data_model2 <- list(
  N = nrow(part1_data),
  R = length(unique(part1_data$region_id)),
  y = part1_data$log_price,
  distance = part1_data$distance_z,
  buildingarea = part1_data$buildingarea_z,
  region = part1_data$region_id
)
```

Model 1: Stan model (method 1)

fitting for STAN Model

```
mod1 <- cmdstan_model("Model1.stan")

fit_m1 <- mod1$sample(
  data = stan_data_model1,
  chains = 4,
  parallel_chains = 4,
  iter_warmup = 1000,
  iter_sampling = 1000,
  seed = 123,
  init = 0,
  refresh = 0,
  adapt_delta = 0.95,
  max_treedepth = 12
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 finished in 12.0 seconds.
## Chain 3 finished in 12.1 seconds.
## Chain 2 finished in 13.8 seconds.
## Chain 4 finished in 14.8 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 13.2 seconds.
## Total execution time: 15.0 seconds.
```

posteriror summaries

```
summ_m1 <- fit_m1$summary(variables = c("beta0", "beta1", "beta2", "sigma")) %>%
  as.data.frame() %>%
  dplyr::select(variable, mean, median, sd, q5, q95, rhat, ess_bulk, ess_tail)

summ_m1
```

```
##   variable        mean      median          sd          q5         q95     rhat
## 1    beta0 13.73941273 13.73921300 0.006139360 13.72925830 13.74960830 1.005012
## 2    beta1 -0.09291254 -0.09300522 0.006276566 -0.10294222 -0.08262237 1.001508
## 3    beta2  0.06173576  0.06167671 0.006555387  0.05105858  0.07252791 1.000429
## 4    sigma  0.53106841  0.53102125 0.004452695  0.52371207  0.53844117 1.001245
##   ess_bulk ess_tail
## 1 4919.469 2946.141
## 2 5556.926 2903.859
## 3 5112.316 2274.807
## 4 4517.947 2801.776
```

Stan sampler diagnostics 1 (Checks divergences and treedepth)

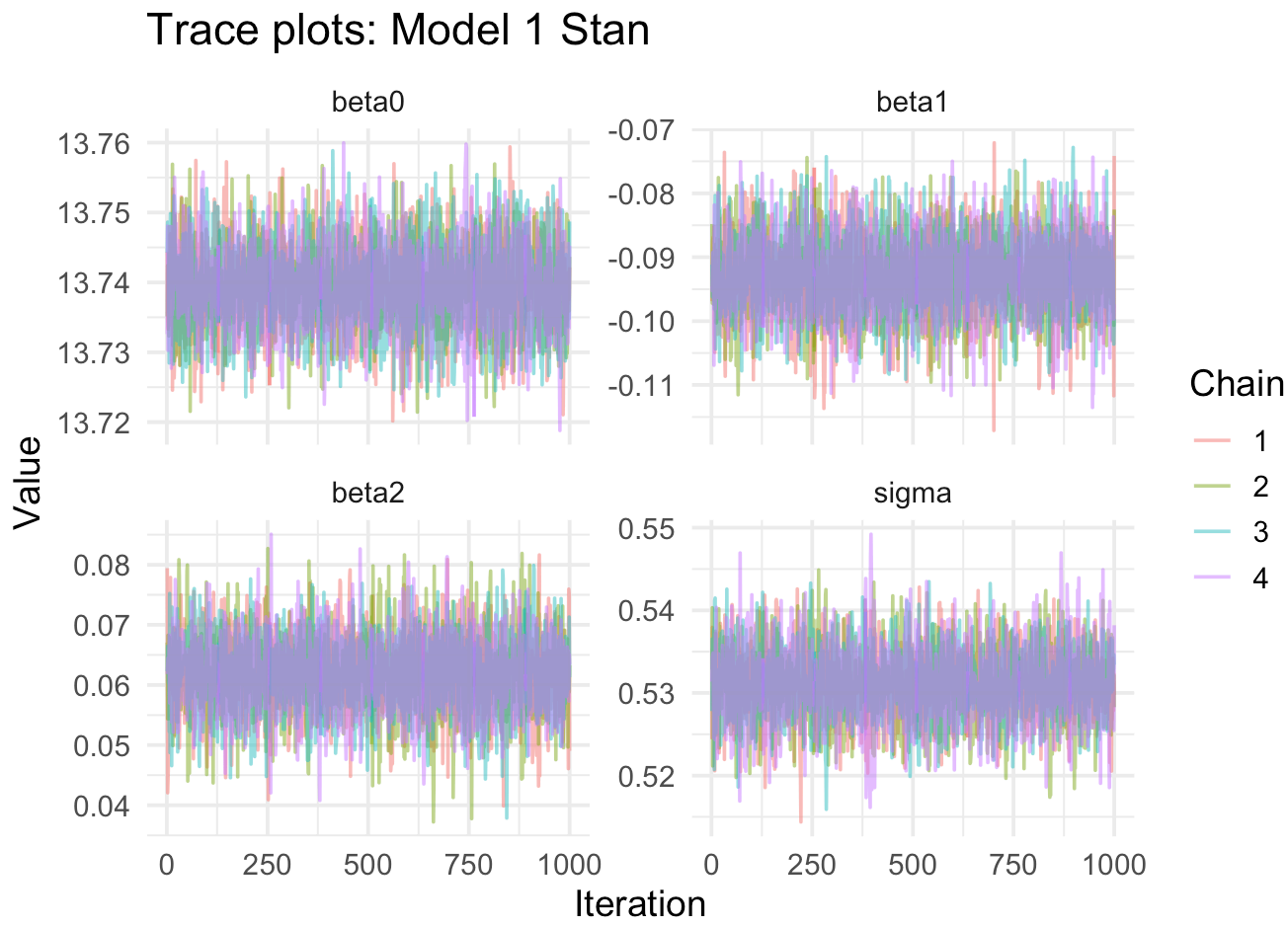
```
check_sampler_issues <- function(fit, treedepth_limit = 12) {
  sdg <- fit$sampler_diagnostics()
  data.frame(
    divergent = sum(sdg[, , "divergent__"]),
    hit_max_treedepth = sum(sdg[, , "treedepth__"] >= treedepth_limit),
    max_treedepth_seen = max(sdg[, , "treedepth__"])
  )
}

check_sampler_issues(fit_m1, 12)
```

```
##   divergent hit_max_treedepth max_treedepth_seen
## 1         0                 0                  3
```

Trace plot

```
fit_m1$draws(variables = c("beta0", "beta1", "beta2", "sigma"), format = "draws_df") %>%
  dplyr::select(.chain, .iteration, beta0, beta1, beta2, sigma) %>%
  pivot_longer(cols = c(beta0, beta1, beta2, sigma), names_to = "parameter", values_to =
"value") %>%
  ggplot(aes(x = .iteration, y = value, colour = factor(.chain))) +
  geom_line(alpha = 0.5) +
  facet_wrap(~ parameter, scales = "free_y", ncol = 2) +
  labs(title = "Trace plots: Model 1 Stan", x = "Iteration", y = "Value", colour = "Chai
n") +
  theme_minimal(base_size = 14)
```

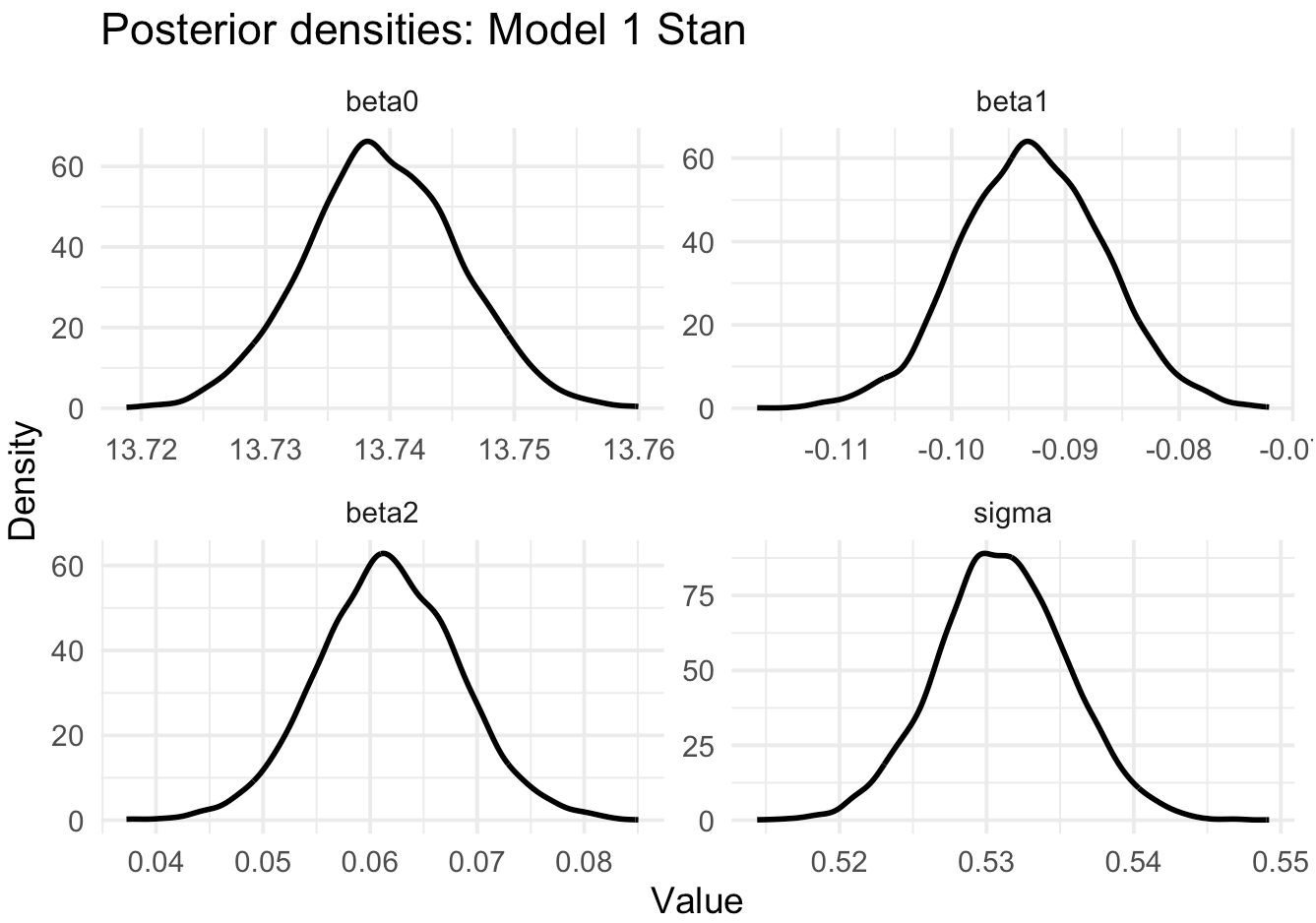
```
## Warning: Dropping 'draws_df' class as required metadata was removed.
```



Posterior density plot (mixing diagnostics)

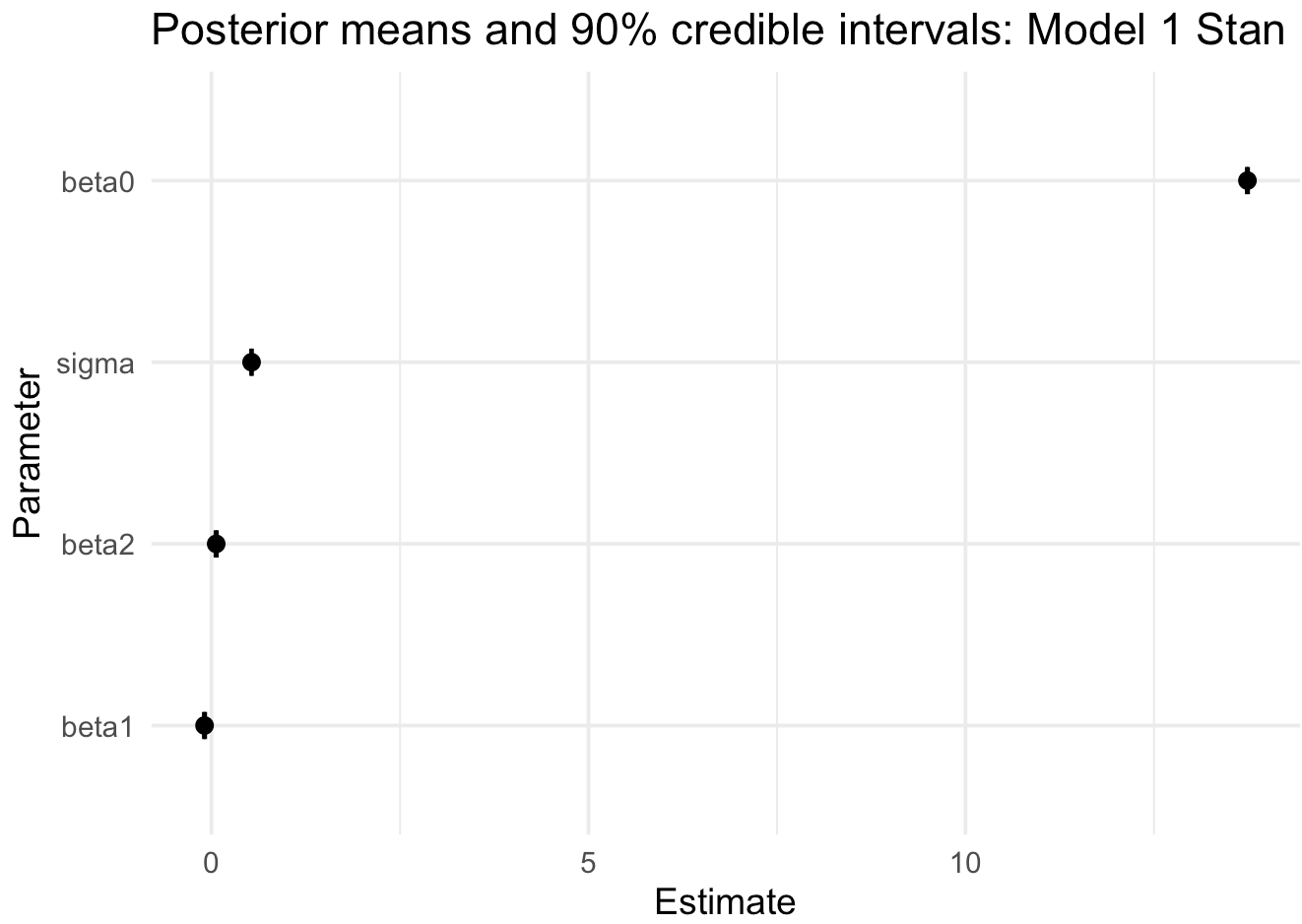
```
fit_m1$draws(variables = c("beta0", "beta1", "beta2", "sigma"), format = "draws_df") %>%
  dplyr::select(beta0, beta1, beta2, sigma) %>%
  pivot_longer(cols = everything(), names_to = "parameter", values_to = "value") %>%
  ggplot(aes(x = value)) +
  geom_density(linewidth = 1) +
  facet_wrap(~ parameter, scales = "free", ncol = 2) +
  labs(title = "Posterior densities: Model 1 Stan", x = "Value", y = "Density") +
  theme_minimal(base_size = 14)
```

```
## Warning: Dropping 'draws_df' class as required metadata was removed.
```



Credible intervals for Model 1 Stan

```
ggplot(summ_m1, aes(x = reorder(variable, mean), y = mean)) +
  geom_point(size = 2.5) +
  geom_errorbar(aes(ymin = q5, ymax = q95), width = 0.15) +
  coord_flip() +
  labs(title = "Posterior means and 90% credible intervals: Model 1 Stan",
       x = "Parameter", y = "Estimate") +
  theme_minimal(base_size = 14)
```



Prior Sensitivity check - robustness check on prior choices

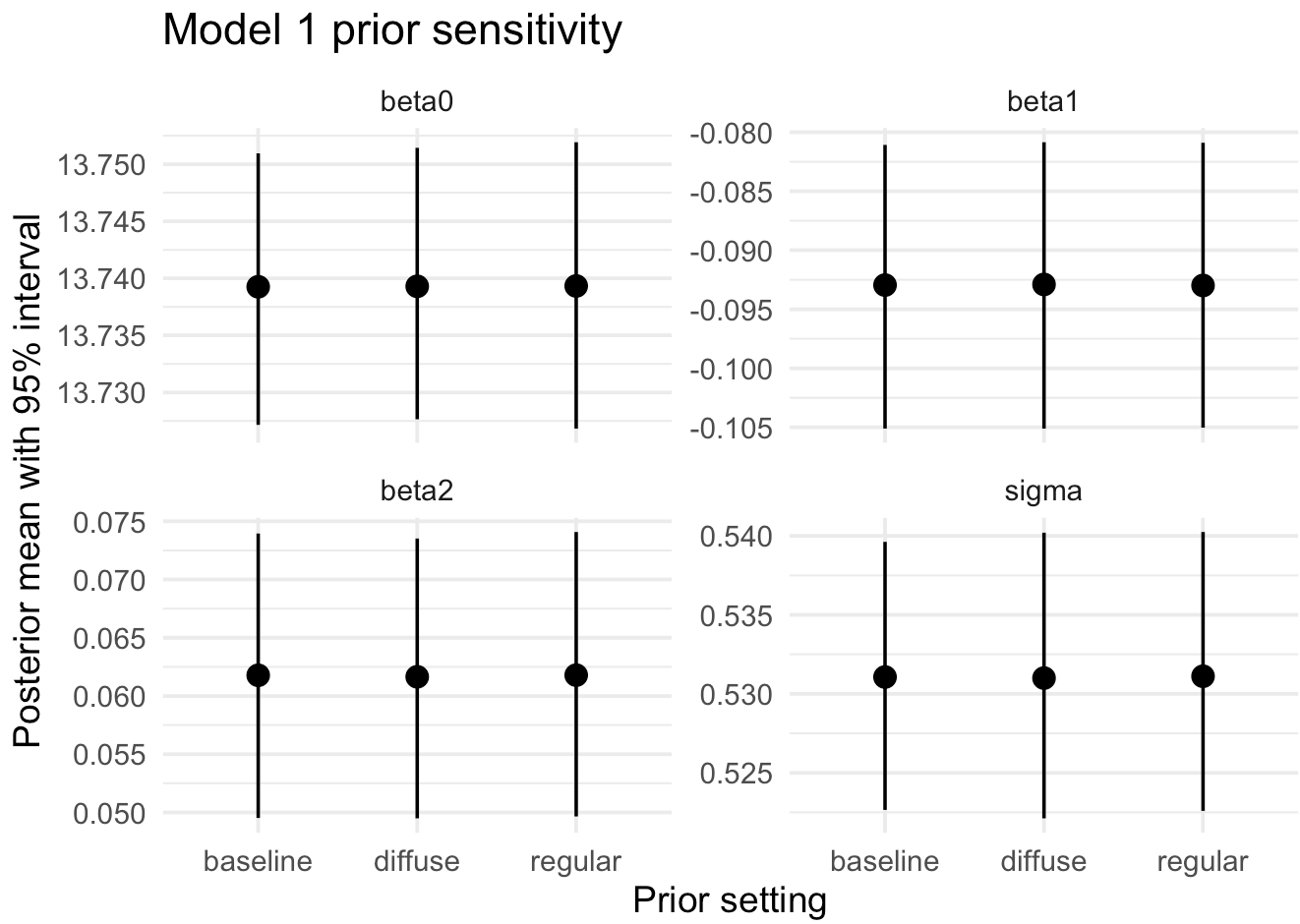
```
mod1_sens <- cmdstan_model("model1_sensitivity.stan")

prior_settings_m1 <- list(
  baseline = list(beta0_sd = 10, beta_sd = 1, sigma_rate = 1),
  diffuse  = list(beta0_sd = 20, beta_sd = 2, sigma_rate = 0.5),
  regular  = list(beta0_sd = 5,  beta_sd = 0.5, sigma_rate = 2)
)

fits_m1_sens <- lapply(prior_settings_m1, function(pr) {
  mod1_sens$sample(
    data = c(
      list(
        N = stan_data_model1$N,
        y = stan_data_model1$y,
        x_dist = stan_data_model1$distance,
        x_area = stan_data_model1$buildingarea
      ),
      pr
    ),
    chains = 4,
    parallel_chains = 4,
    iter_warmup = 1000,
    iter_sampling = 1000,
    seed = 123,
    refresh = 0
  )
})
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 finished in 8.0 seconds.
## Chain 2 finished in 8.1 seconds.
## Chain 4 finished in 8.0 seconds.
## Chain 3 finished in 8.4 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 8.2 seconds.
## Total execution time: 8.5 seconds.
##
## Running MCMC with 4 parallel chains...
##
## Chain 1 finished in 7.1 seconds.
## Chain 3 finished in 7.1 seconds.
## Chain 2 finished in 7.5 seconds.
## Chain 4 finished in 7.6 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 7.3 seconds.
## Total execution time: 7.7 seconds.
##
## Running MCMC with 4 parallel chains...
##
## Chain 2 finished in 7.3 seconds.
## Chain 3 finished in 7.5 seconds.
## Chain 4 finished in 7.7 seconds.
## Chain 1 finished in 7.8 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 7.6 seconds.
## Total execution time: 7.9 seconds.
```

```
summarize_fit_m1 <- function(fit, prior_name) {
  draws <- fit$draws(variables = c("beta0", "beta1", "beta2", "sigma"), format = "draws_
df")
  dplyr::bind_rows(
    data.frame(prior = prior_name, parameter = "beta0", mean = mean(draws$beta0), q2.5 =
quantile(draws$beta0, 0.025), q97.5 = quantile(draws$beta0, 0.975)),
    data.frame(prior = prior_name, parameter = "beta1", mean = mean(draws$beta1), q2.5 =
quantile(draws$beta1, 0.025), q97.5 = quantile(draws$beta1, 0.975)),
    data.frame(prior = prior_name, parameter = "beta2", mean = mean(draws$beta2), q2.5 =
quantile(draws$beta2, 0.025), q97.5 = quantile(draws$beta2, 0.975)),
    data.frame(prior = prior_name, parameter = "sigma", mean = mean(draws$sigma), q2.5 =
quantile(draws$sigma, 0.025), q97.5 = quantile(draws$sigma, 0.975))
  )
}

sens_table_m1 <- dplyr::bind_rows(
  summarize_fit_m1(fits_m1_sens$baseline, "baseline"),
  summarize_fit_m1(fits_m1_sens$diffuse, "diffuse"),
  summarize_fit_m1(fits_m1_sens$regular, "regular")
)

sens_table_m1
```

```
##              prior parameter        mean        q2.5       q97.5
## 2.5%...1  baseline     beta0 13.73925948 13.72715770 13.75095115
## 2.5%...2  baseline     beta1 -0.09294977 -0.10510790 -0.08107133
## 2.5%...3  baseline     beta2  0.06177745  0.04952635  0.07395435
## 2.5%...4  baseline     sigma  0.53106662  0.52265500  0.53962361
## 2.5%...5   diffuse     beta0 13.73930046 13.72764657 13.75143915
## 2.5%...6   diffuse     beta1 -0.09289619 -0.10510979 -0.08084829
## 2.5%...7   diffuse     beta2  0.06164919  0.04948932  0.07352390
## 2.5%...8   diffuse     sigma  0.53099960  0.52211754  0.54020308
## 2.5%...9   regular     beta0 13.73933000 13.72682960 13.75192708
## 2.5%...10  regular     beta1 -0.09298413 -0.10502744 -0.08089224
## 2.5%...11  regular     beta2  0.06178984  0.04965204  0.07408357
## 2.5%...12  regular     sigma  0.53111459  0.52259387  0.54024698
```

```
ggplot(sens_table_m1, aes(x = prior, y = mean, ymin = q2.5, ymax = q97.5)) +
  geom_pointrange() +
  facet_wrap(~ parameter, scales = "free_y") +
  labs(title = "Model 1 prior sensitivity", x = "Prior setting", y = "Posterior mean wit
h 95% interval") +
  theme_minimal(base_size = 14)
```



Model 1 - Custom MH (method 2)

Define model 1 with our target log posterior

```
log_posterior_model1 <- function(theta, y, distance, buildingarea) {
  beta0 <- theta[1]
  beta1 <- theta[2]
  beta2 <- theta[3]
  eta <- theta[4]
  sigma <- exp(eta)

  mu <- beta0 + beta1 * distance + beta2 * buildingarea

  log_prior <-
    dnorm(beta0, 0, 10, log = TRUE) +
    dnorm(beta1, 0, 1, log = TRUE) +
    dnorm(beta2, 0, 1, log = TRUE) +
    dexp(sigma, rate = 1, log = TRUE) + eta

  log_like <- sum(dnorm(y, mean = mu, sd = sigma, log = TRUE))

  log_prior + log_like
}
```

Create MH update and sampling functions, basically is a setup for custom posterior sample

```
mh_step_with_lp <- function(current, current_lp, proposal_sds, y, distance, buildingare
a, lp_fun) {
  proposal <- current + rnorm(length(current), mean = 0, sd = proposal_sds)
  proposal_lp <- lp_fun(proposal, y, distance, buildingarea)

  log_alpha <- proposal_lp - current_lp
  accepted <- log(runif(1)) < log_alpha

  if (accepted) {
    list(state = proposal, lp = proposal_lp, accepted = TRUE)
  } else {
    list(state = current, lp = current_lp, accepted = FALSE)
  }
}

mh_mixture_with_lp <- function(y, distance, buildingarea, init, lp_fun,
                               n_iter = 30000,
                               proposal_sds_small = c(0.01, 0.001, 0.001, 0.01),
                               proposal_sds_large = c(0.03, 0.003, 0.003, 0.03),
                               mix_prob_small = 0.7) {
  samples <- matrix(NA_real_, nrow = n_iter, ncol = 4)
  colnames(samples) <- c("beta0", "beta1", "beta2", "eta")

  current <- init
  current_lp <- lp_fun(current, y, distance, buildingarea)
  accepted <- logical(n_iter)

  for (i in seq_len(n_iter)) {
    sds <- if (runif(1) < mix_prob_small) proposal_sds_small else proposal_sds_large
    out <- mh_step_with_lp(current, current_lp, sds, y, distance, buildingarea, lp_fun)
    current <- out$state
    current_lp <- out$lp
    accepted[i] <- out$accepted
    samples[i, ] <- current
  }

  list(samples = samples, acceptance_rate = mean(accepted))
}
```

Fitting

```
set.seed(123)

mh_draws_list <- vector("list", 4)
acc_rates <- numeric(4)

for (ch in 1:4) {
  out <- mh_mixture_with_lp(
    y = stan_data_model1$y,
    distance = stan_data_model1$distance,
    buildingarea = stan_data_model1$buildingarea,
    init = c(13.5, -0.1, 0.05, log(0.5)) + rnorm(4, 0, 0.05),
    lp_fun = log_posterior_model1,
    n_iter = 30000
  )

  draws <- as.data.frame(out$samples)
  draws$sigma <- exp(draws$eta)
  draws$iter <- 1:nrow(draws)
  draws$chain <- ch

  mh_draws_list[[ch]] <- draws
  acc_rates[ch] <- out$acceptance_rate
}

acc_rates
```

```
## [1] 0.3213000 0.3258333 0.3178333 0.3199000
```

```
mean(acc_rates)
```

```
## [1] 0.3212167
```

Burn-in for MH draws

```
burn_in_m1 <- 15000

mh_post_list <- lapply(mh_draws_list, function(df) {
  df %>% filter(iter > burn_in_m1)
})

mh_all_draws <- dplyr::bind_rows(mh_post_list)
head(mh_all_draws)
```

```
##      beta0       beta1      beta2        eta     sigma  iter chain
## 1 13.73938 -0.09744554 0.07457035 -0.6356742 0.5295783 15001     1
## 2 13.73938 -0.09744554 0.07457035 -0.6356742 0.5295783 15002     1
## 3 13.73938 -0.09744554 0.07457035 -0.6356742 0.5295783 15003     1
## 4 13.73938 -0.09744554 0.07457035 -0.6356742 0.5295783 15004     1
## 5 13.73938 -0.09744554 0.07457035 -0.6356742 0.5295783 15005     1
## 6 13.73938 -0.09744554 0.07457035 -0.6356742 0.5295783 15006     1
```

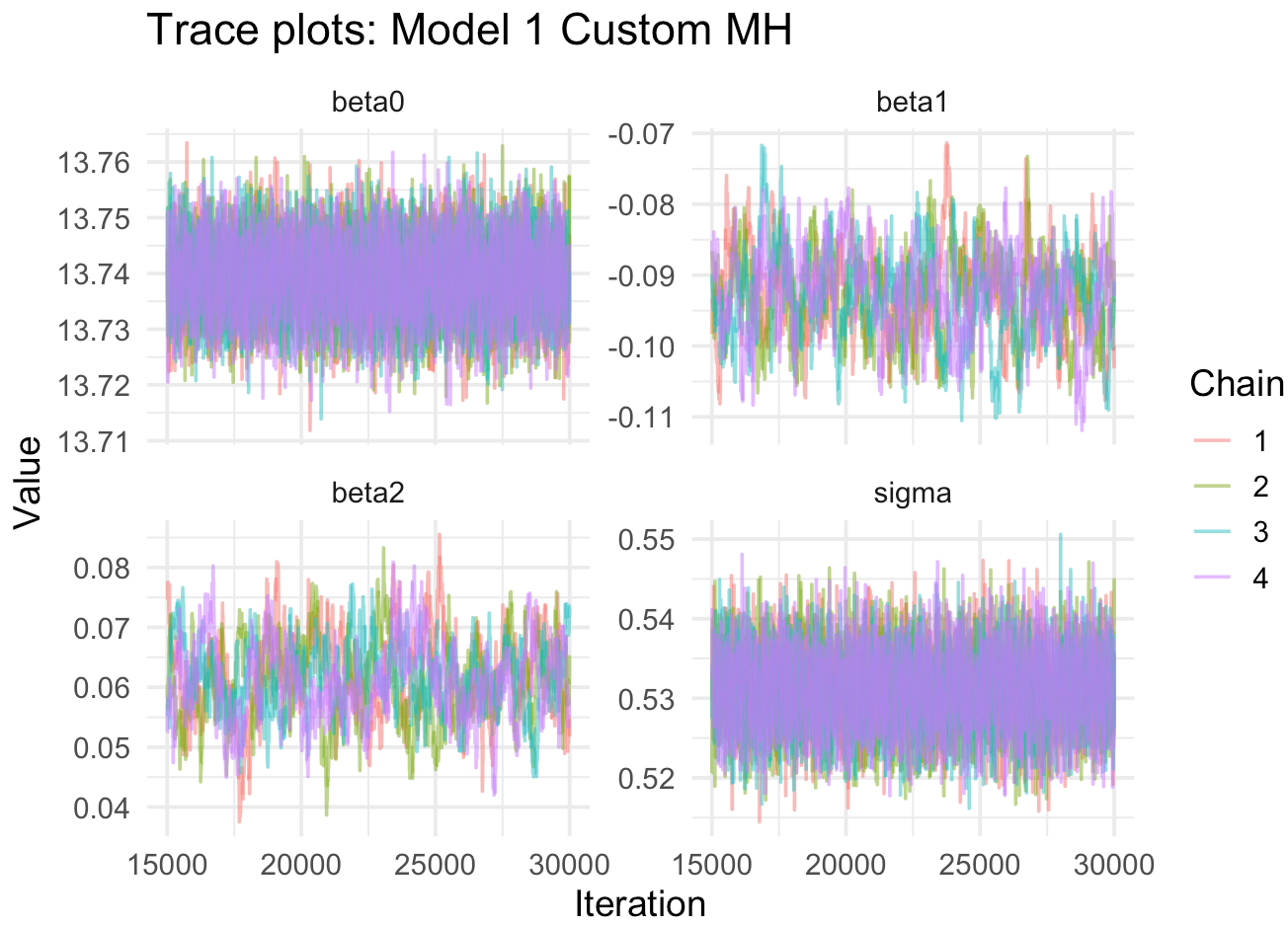
Posterior summaries for MH

```
mh_m1_summary <- mh_all_draws %>%
  summarise(
    beta0_mean = mean(beta0), beta1_mean = mean(beta1), beta2_mean = mean(beta2), sigma_
mean = mean(sigma),
    beta0_median = median(beta0), beta1_median = median(beta1), beta2_median = median(be
ta2), sigma_median = median(sigma),
    beta0_sd = sd(beta0), beta1_sd = sd(beta1), beta2_sd = sd(beta2), sigma_sd = sd(sigm
a),
    beta0_q5 = quantile(beta0, 0.05), beta1_q5 = quantile(beta1, 0.05), beta2_q5 = quant
ile(beta2, 0.05), sigma_q5 = quantile(sigma, 0.05),
    beta0_q95 = quantile(beta0, 0.95), beta1_q95 = quantile(beta1, 0.95), beta2_q95 = qu
antile(beta2, 0.95), sigma_q95 = quantile(sigma, 0.95)
  ) %>%
  pivot_longer(cols = everything(), names_to = c("variable", ".value"), names_sep = "_")

mh_m1_summary
```

```
## # A tibble: 4 × 6
##   variable    mean  median      sd      q5     q95
##   <chr>      <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
## 1 beta0    13.7    13.7    0.00627 13.7    13.7
## 2 beta1    -0.0929 -0.0927 0.00603 -0.103  -0.0830
## 3 beta2     0.0613  0.0611 0.00653  0.0508  0.0723
## 4 sigma     0.531   0.531  0.00447  0.524   0.538
```

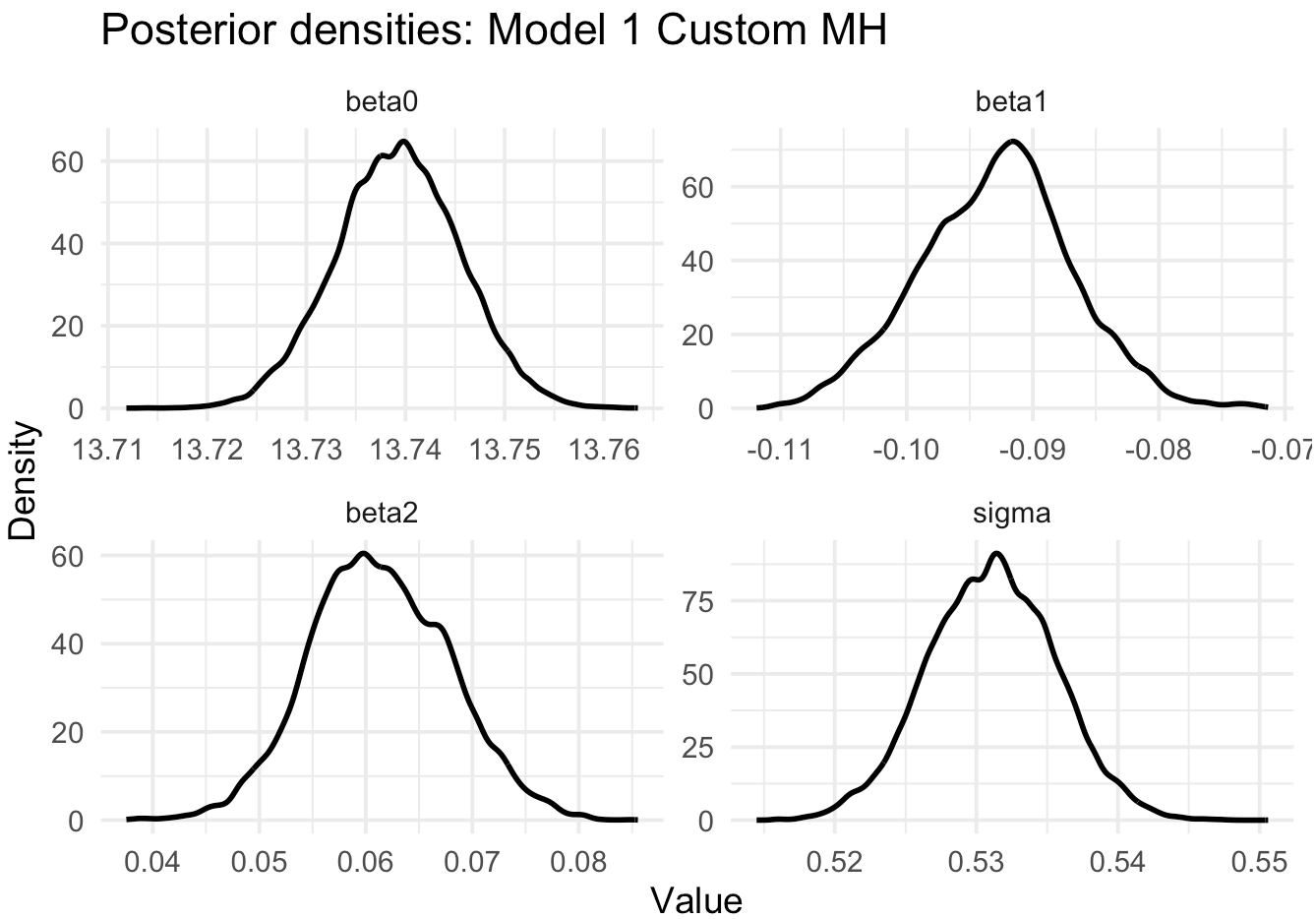
Trace plot

```
mh_all_draws %>%
  dplyr::select(iter, chain, beta0, beta1, beta2, sigma) %>%
  pivot_longer(cols = c(beta0, beta1, beta2, sigma), names_to = "parameter", values_to =
"value") %>%
  ggplot(aes(x = iter, y = value, colour = factor(chain))) +
  geom_line(alpha = 0.5) +
  facet_wrap(~ parameter, scales = "free_y", ncol = 2) +
  labs(title = "Trace plots: Model 1 Custom MH", x = "Iteration", y = "Value", colour =
"Chain") +
  theme_minimal(base_size = 14)
```



Posterior densities for model 1 mh

```
mh_all_draws %>%
  dplyr::select(beta0, beta1, beta2, sigma) %>%
  pivot_longer(cols = everything(), names_to = "parameter", values_to = "value") %>%
  ggplot(aes(x = value)) +
  geom_density(linewidth = 1) +
  facet_wrap(~ parameter, scales = "free", ncol = 2) +
  labs(title = "Posterior densities: Model 1 Custom MH", x = "Value", y = "Density") +
  theme_minimal(base_size = 14)
```



MH convergence diagnostics

(used to compute for Gelman-Rubin and effective sample size)

```
mh_chain_list <- lapply(mh_post_list, function(df) {
  coda::mcmc(df[, c("beta0", "beta1", "beta2", "sigma")])
})

mh_mcmc_list <- coda::mcmc.list(mh_chain_list)

coda::gelman.diag(mh_mcmc_list, autoburnin = FALSE)
```

```
## Potential scale reduction factors:
##
##       Point est. Upper C.I.
## beta0       1.00       1.00
## beta1       1.00       1.01
## beta2       1.01       1.03
## sigma       1.00       1.00
##
## Multivariate psrf
##
## 1.01
```

```
coda::effectiveSize(mh_mcmc_list)
```

```
##     beta0     beta1     beta2     sigma
## 7408.6322  214.9354  196.2456 5395.8843
```

Invariance check

```
#out_beta0_bug$ks_beta0
#out_beta1_bug$ks_beta1
#out_beta2_bug$ks_beta2
```

Stan vs Custom MH for model 1

summary comparison table

```
stan_m1_summary <- fit_m1$summary(variables = c("beta0", "beta1", "beta2", "sigma")) %>%
  as.data.frame() %>%
  dplyr::select(variable, mean, median, sd, q5, q95) %>%
  mutate(method = "Stan")

mh_m1_summary2 <- mh_all_draws %>%
  summarise(
    beta0_mean = mean(beta0), beta1_mean = mean(beta1), beta2_mean = mean(beta2), sigma_
mean = mean(sigma),
    beta0_median = median(beta0), beta1_median = median(beta1), beta2_median = median(be
ta2), sigma_median = median(sigma),
    beta0_sd = sd(beta0), beta1_sd = sd(beta1), beta2_sd = sd(beta2), sigma_sd = sd(sigm
a),
    beta0_q5 = quantile(beta0, 0.05), beta1_q5 = quantile(beta1, 0.05), beta2_q5 = quant
ile(beta2, 0.05), sigma_q5 = quantile(sigma, 0.05),
    beta0_q95 = quantile(beta0, 0.95), beta1_q95 = quantile(beta1, 0.95), beta2_q95 = qu
antile(beta2, 0.95), sigma_q95 = quantile(sigma, 0.95)
  ) %>%
  pivot_longer(cols = everything(), names_to = c("variable", ".value"), names_sep = "_")
%>%
  mutate(method = "Custom MH")

comparison_table_m1 <- dplyr::bind_rows(stan_m1_summary, mh_m1_summary2) %>%
  arrange(variable, method)

comparison_table_m1
```

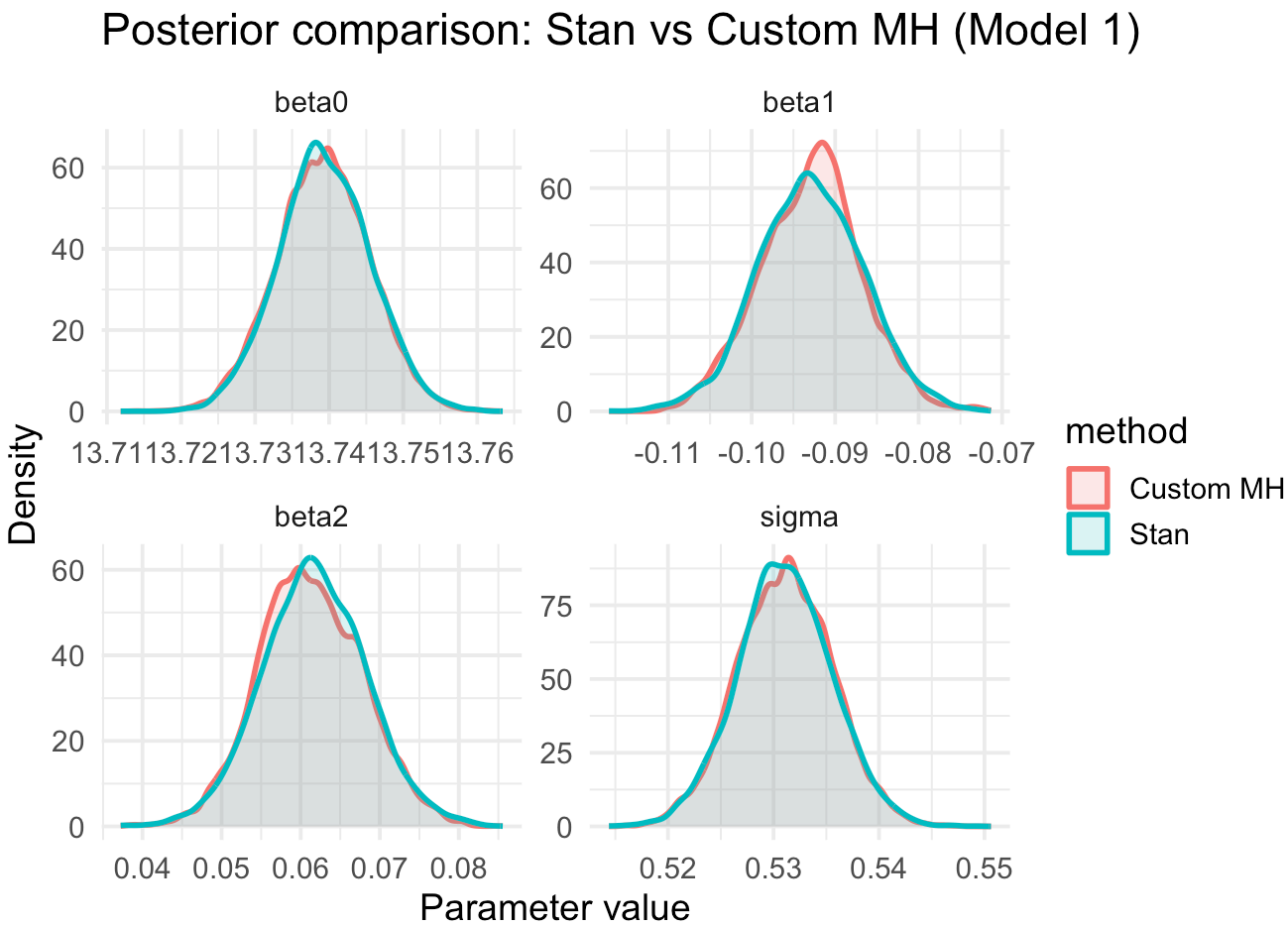
```
##   variable        mean      median          sd          q5         q95
## 1    beta0 13.73922490 13.73929124 0.006268365 13.72885487 13.74950993
## 2    beta0 13.73941273 13.73921300 0.006139360 13.72925830 13.74960830
## 3    beta1 -0.09290203 -0.09266517 0.006033493 -0.10315694 -0.08302731
## 4    beta1 -0.09291254 -0.09300522 0.006276566 -0.10294222 -0.08262237
## 5    beta2  0.06132163  0.06107597 0.006530228  0.05082790  0.07232580
## 6    beta2  0.06173576  0.06167671 0.006555387  0.05105858  0.07252791
## 7    sigma  0.53109012  0.53113184 0.004474404  0.52375775  0.53838352
## 8    sigma  0.53106841  0.53102125 0.004452695  0.52371207  0.53844117
##      method
## 1 Custom MH
## 2      Stan
## 3 Custom MH
## 4      Stan
## 5 Custom MH
## 6      Stan
## 7 Custom MH
## 8      Stan
```

Method comparison check with density overlay

```
plot_draws_m1 <- dplyr::bind_rows(
  mh_all_draws %>% dplyr::select(beta0, beta1, beta2, sigma) %>% mutate(method = "Custom
MH"),
  fit_m1$draws(variables = c("beta0", "beta1", "beta2", "sigma"), format = "df") %>%
    dplyr::select(beta0, beta1, beta2, sigma) %>%
    mutate(method = "Stan")
) %>%
  pivot_longer(cols = c(beta0, beta1, beta2, sigma), names_to = "parameter", values_to =
"value")
```

```
## Warning: Dropping 'draws_df' class as required metadata was removed.
```

```
ggplot(plot_draws_m1, aes(x = value, colour = method, fill = method)) +
  geom_density(alpha = 0.18, linewidth = 1) +
  facet_wrap(~ parameter, scales = "free", ncol = 2) +
  labs(title = "Posterior comparison: Stan vs Custom MH (Model 1)", x = "Parameter valu
e", y = "Density") +
  theme_minimal(base_size = 14)
```



Model 2 with stan

```
mod2 <- cmdstan_model("Part2_Model2_Hierarchical.stan")
fit_m2 <- mod2$sample(
  data = stan_data_model2,
  chains = 4,
  parallel_chains = 4,
  iter_warmup = 1000,
  iter_sampling = 1000,
  seed = 123,
  init = 0,
  refresh = 0,
  adapt_delta = 0.99,
  max_treedepth = 12
)
```

```
## Running MCMC with 4 parallel chains...
```

```
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 1 Exception: normal_lpdf: Location parameter[1] is inf, but must be finite! (in
'/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan', l
ine 36, column 2 to line 39, column 4)
```

```
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 1 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 1
```

```
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 2 Exception: normal_lpdf: Location parameter[1] is inf, but must be finite! (in
'/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan', l
ine 36, column 2 to line 39, column 4)
```

```
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 2 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 2
```

```
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 3 Exception: normal_lpdf: Location parameter[1] is inf, but must be finite! (in
'/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan', l
ine 36, column 2 to line 39, column 4)
```

```
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 3 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 3
```

```
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 4 Exception: normal_lpdf: Location parameter[1] is inf, but must be finite! (in
'/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan', l
ine 36, column 2 to line 39, column 4)
```

```
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 4 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 4
```

```
## Chain 3 finished in 186.4 seconds.
## Chain 1 finished in 250.7 seconds.
## Chain 4 finished in 258.1 seconds.
## Chain 2 finished in 389.3 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 271.1 seconds.
## Total execution time: 389.5 seconds.
```

Posterior summaries

```
summ_m2 <- fit_m2$summary(variables = c("beta0", "beta1", "beta2", "tau", "sigma")) %>%
  as.data.frame() %>%
  dplyr::select(variable, mean, median, sd, q5, q95, rhat, ess_bulk, ess_tail)

summ_m2
```

```
##   variable        mean      median          sd         q5         q95      rhat
## 1    beta0 13.71747628 13.71995600 0.110217099 13.5426936 13.88709115 1.0038519
## 2    beta1 -0.11998924 -0.11996732 0.007606580 -0.1324753 -0.10751567 1.0008578
## 3    beta2  0.06160055  0.06158543 0.005964133  0.0518828  0.07130673 1.0013980
## 4      tau  0.27953124  0.25520837 0.103461961  0.1636727  0.47580272 1.0049411
## 5    sigma  0.49466125  0.49459381 0.004128923  0.4878721  0.50133496 0.9999252
##    ess_bulk ess_tail
## 1  912.5302 1286.361
## 2 3024.2839 2406.998
## 3 3653.8845 2615.588
## 4  878.3615 1307.842
## 5 3808.3166 2560.352
```

Sampler diagnostics

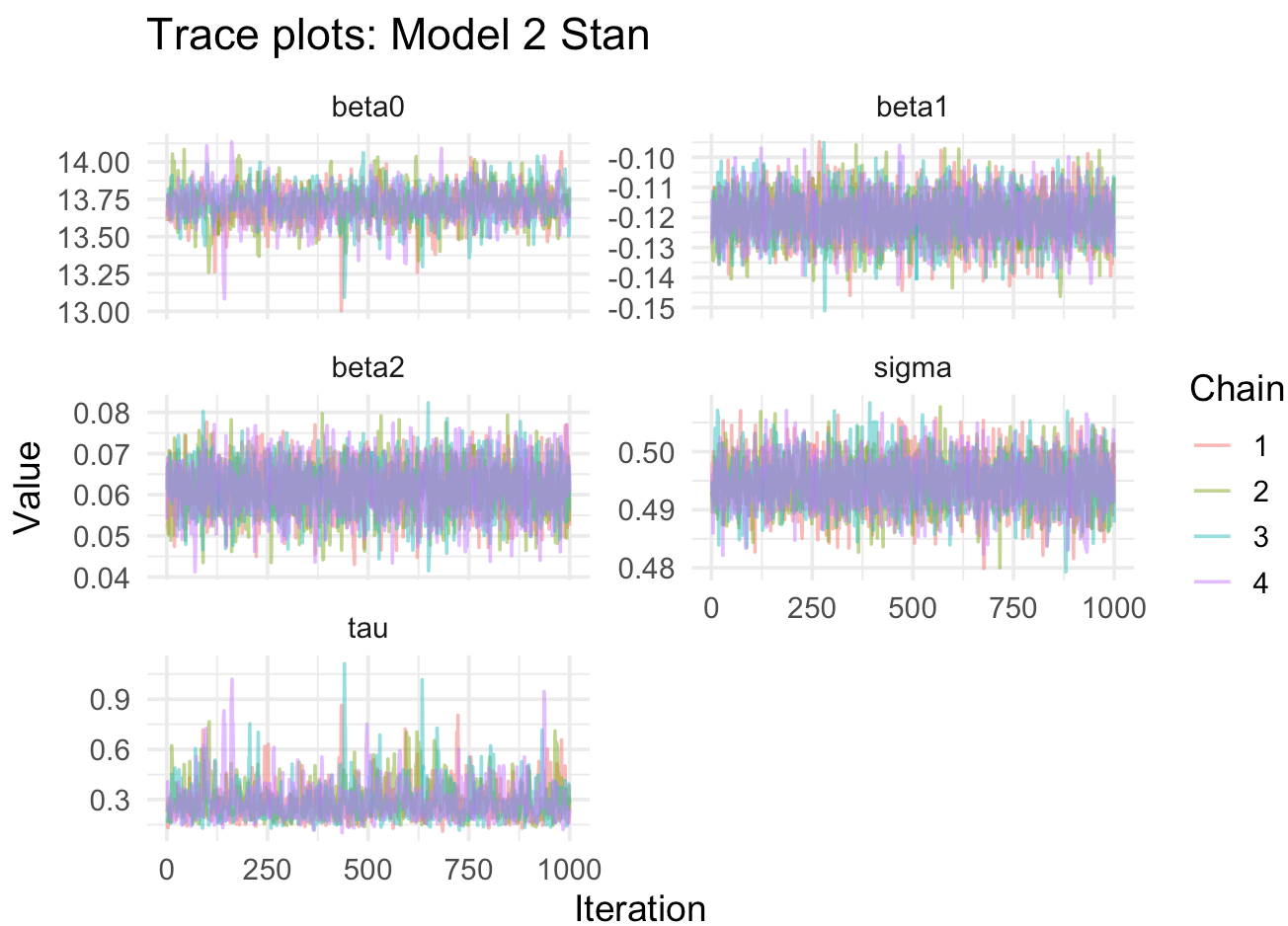
```
check_sampler_issues(fit_m2, 12)
```

```
##   divergent hit_max_treedepth max_treedepth_seen
## 1         0                 0                  9
```

Trace plot

```
fit_m2$draws(variables = c("beta0", "beta1", "beta2", "tau", "sigma"), format = "draws_d
f") %>%
  dplyr::select(.chain, .iteration, beta0, beta1, beta2, tau, sigma) %>%
  pivot_longer(cols = c(beta0, beta1, beta2, tau, sigma), names_to = "parameter", values
_to = "value") %>%
  ggplot(aes(x = .iteration, y = value, colour = factor(.chain))) +
  geom_line(alpha = 0.5) +
  facet_wrap(~ parameter, scales = "free_y", ncol = 2) +
  labs(title = "Trace plots: Model 2 Stan", x = "Iteration", y = "Value", colour = "Chai
n") +
  theme_minimal(base_size = 14)
```

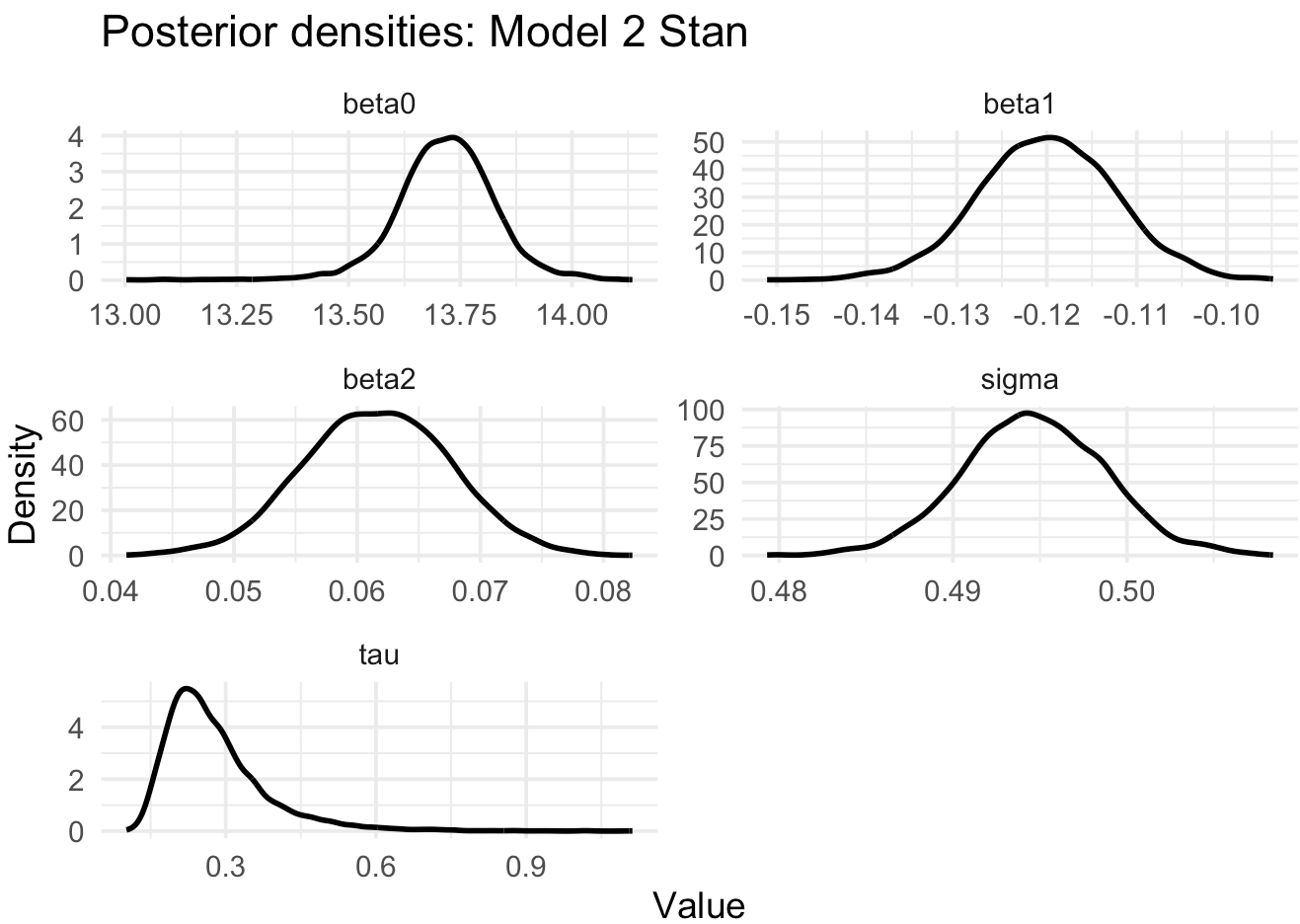
```
## Warning: Dropping 'draws_df' class as required metadata was removed.
```



Posterior densities

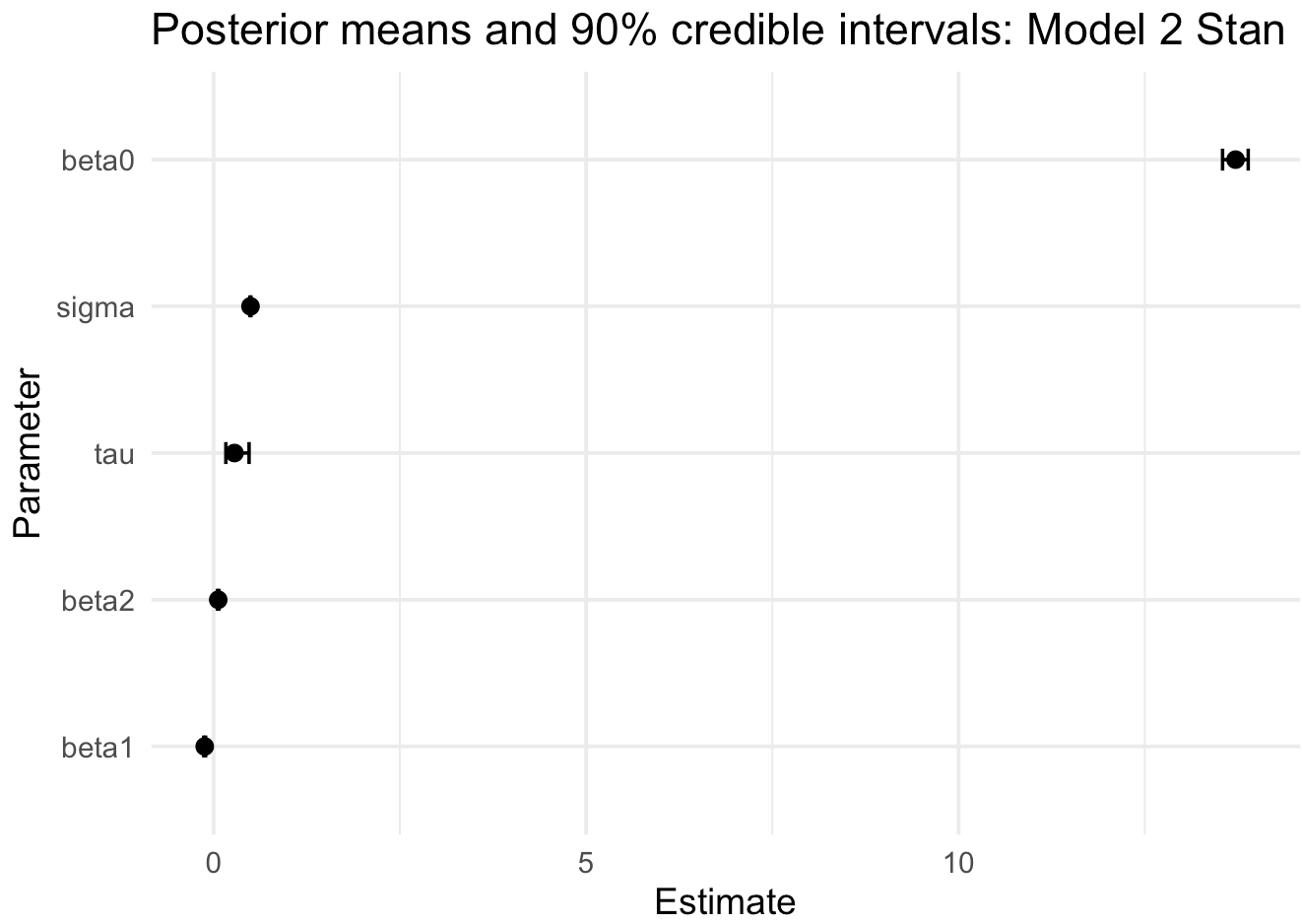
```
fit_m2$draws(variables = c("beta0", "beta1", "beta2", "tau", "sigma"), format = "draws_d
f") %>%
  dplyr::select(beta0, beta1, beta2, tau, sigma) %>%
  pivot_longer(cols = everything(), names_to = "parameter", values_to = "value") %>%
  ggplot(aes(x = value)) +
  geom_density(linewidth = 1) +
  facet_wrap(~ parameter, scales = "free", ncol = 2) +
  labs(title = "Posterior densities: Model 2 Stan", x = "Value", y = "Density") +
  theme_minimal(base_size = 14)
```

```
## Warning: Dropping 'draws_df' class as required metadata was removed.
```



Credible intervals for model 2 stan

```
ggplot(summ_m2, aes(x = reorder(variable, mean), y = mean)) +
  geom_point(size = 2.5) +
  geom_errorbar(aes(ymin = q5, ymax = q95), width = 0.15) +
  coord_flip() +
  labs(title = "Posterior means and 90% credible intervals: Model 2 Stan",
       x = "Parameter", y = "Estimate") +
  theme_minimal(base_size = 14)
```

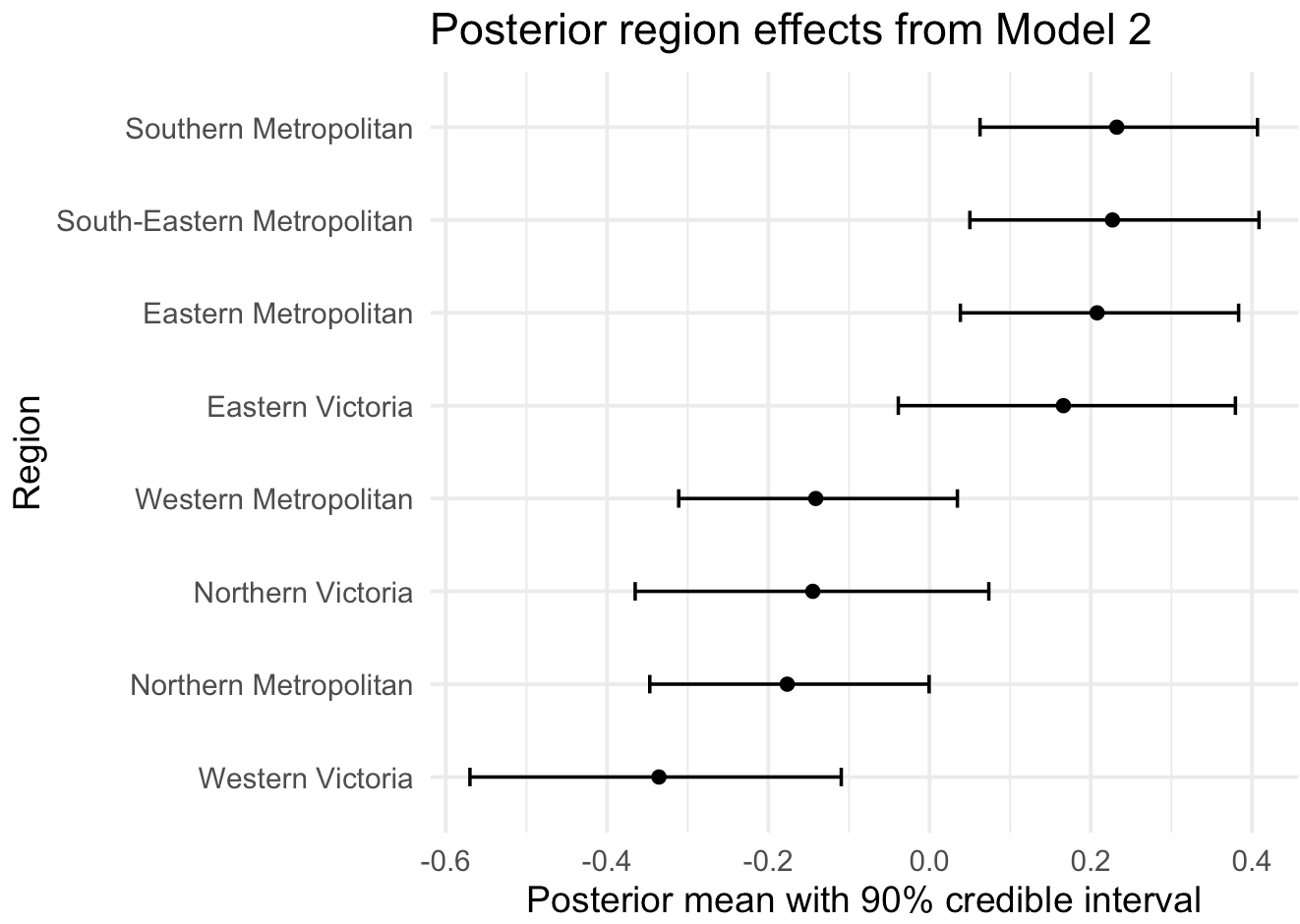


Regional effects from model 2 - interpetation

```
region_levels <- levels(as.factor(part1_data$Regionname))

u_summary <- fit_m2$summary(variables = "u") %>%
  as.data.frame() %>%
  mutate(region = region_levels) %>%
  dplyr::select(region, mean, median, sd, q5, q95, rhat, ess_bulk, ess_tail)

u_summary
```

```
##                       region       mean     median        sd          q5
## 1       Eastern Metropolitan  0.2077848  0.2052650 0.1105536  0.03822098
## 2           Eastern Victoria  0.1660007  0.1623755 0.1334132 -0.03874269
## 3      Northern Metropolitan -0.1765452 -0.1794792 0.1111621 -0.34696413
## 4          Northern Victoria -0.1448075 -0.1453564 0.1360984 -0.36509980
## 5 South-Eastern Metropolitan  0.2268947  0.2252371 0.1135596  0.04995826
## 6      Southern Metropolitan  0.2322065  0.2303012 0.1106278  0.06261316
## 7       Western Metropolitan -0.1411592 -0.1429946 0.1107944 -0.31115321
## 8           Western Victoria -0.3356407 -0.3331150 0.1420755 -0.57004643
##             q95     rhat  ess_bulk ess_tail
## 1  0.3833622155 1.003886  957.6953 1355.089
## 2  0.3793346070 1.002101 1284.7892 1932.969
## 3 -0.0005362266 1.003441  922.3570 1307.859
## 4  0.0735006533 1.001137 1331.6841 1600.797
## 5  0.4085654130 1.003560  896.3971 1424.202
## 6  0.4067290780 1.003911  916.0481 1256.467
## 7  0.0345254873 1.003898  910.2813 1290.535
## 8 -0.1095083160 1.002369 1451.6495 2257.010
```

```
ggplot(u_summary, aes(x = reorder(region, mean), y = mean)) +
  geom_point() +
  geom_errorbar(aes(ymin = q5, ymax = q95), width = 0.2) +
  coord_flip() +
  labs(title = "Posterior region effects from Model 2",
       x = "Region", y = "Posterior mean with 90% credible interval") +
  theme_minimal(base_size = 14)
```

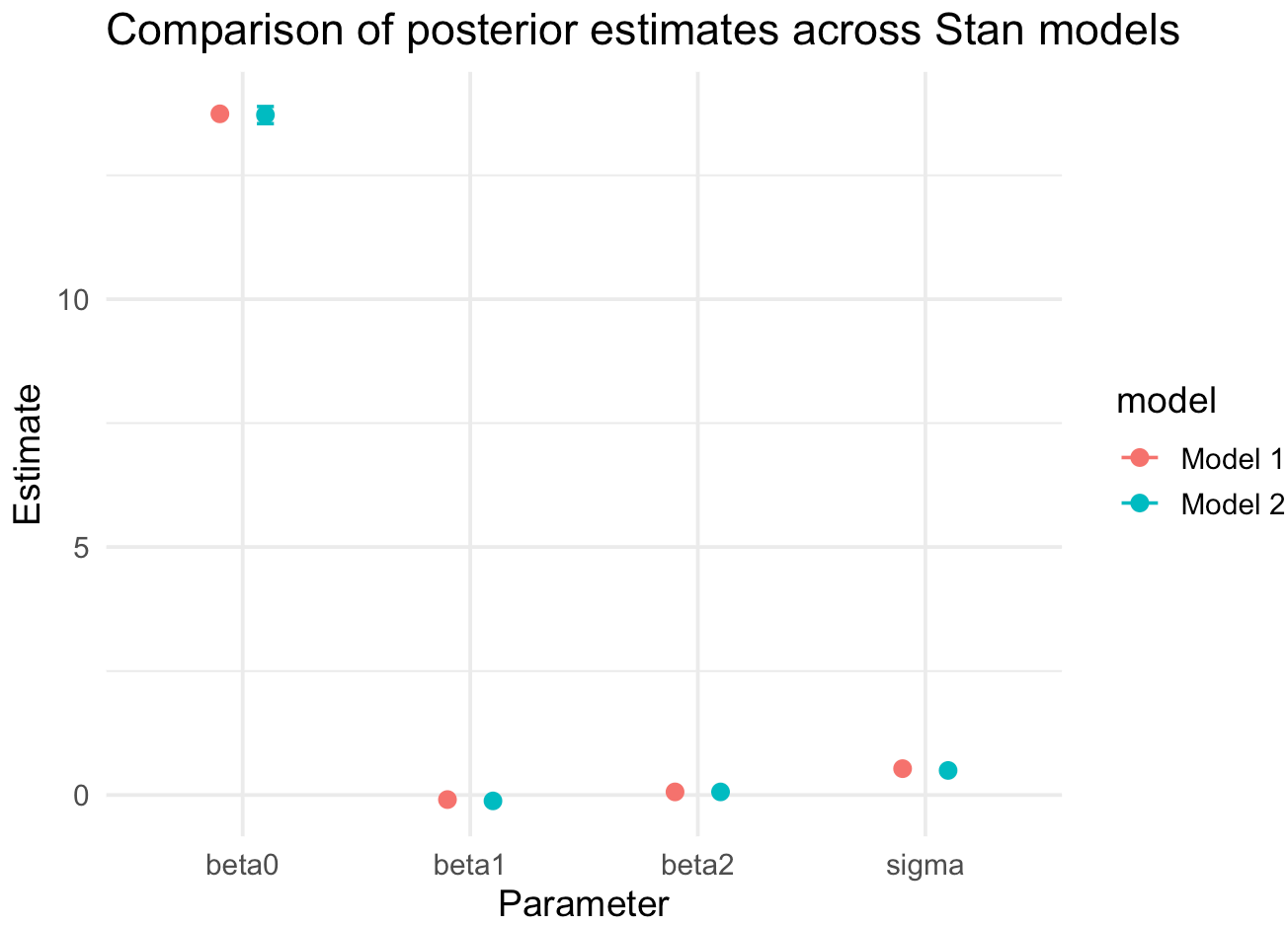


Sensitivity check for model 2

Model 1 vs Model 2

compare interval across model

```
compare_plot_df <- dplyr::bind_rows(
  summ_m1 %>% mutate(model = "Model 1"),
  summ_m2 %>% mutate(model = "Model 2")
) %>%
  filter(variable %in% c("beta0", "beta1", "beta2", "sigma"))

ggplot(compare_plot_df, aes(x = variable, y = mean, colour = model)) +
  geom_point(position = position_dodge(width = 0.4), size = 2.5) +
  geom_errorbar(aes(ymin = q5, ymax = q95), position = position_dodge(width = 0.4), widt
h = 0.15) +
  labs(title = "Comparison of posterior estimates across Stan models",
       x = "Parameter", y = "Estimate") +
  theme_minimal(base_size = 14)
```



Model adequacy and predictive comparison

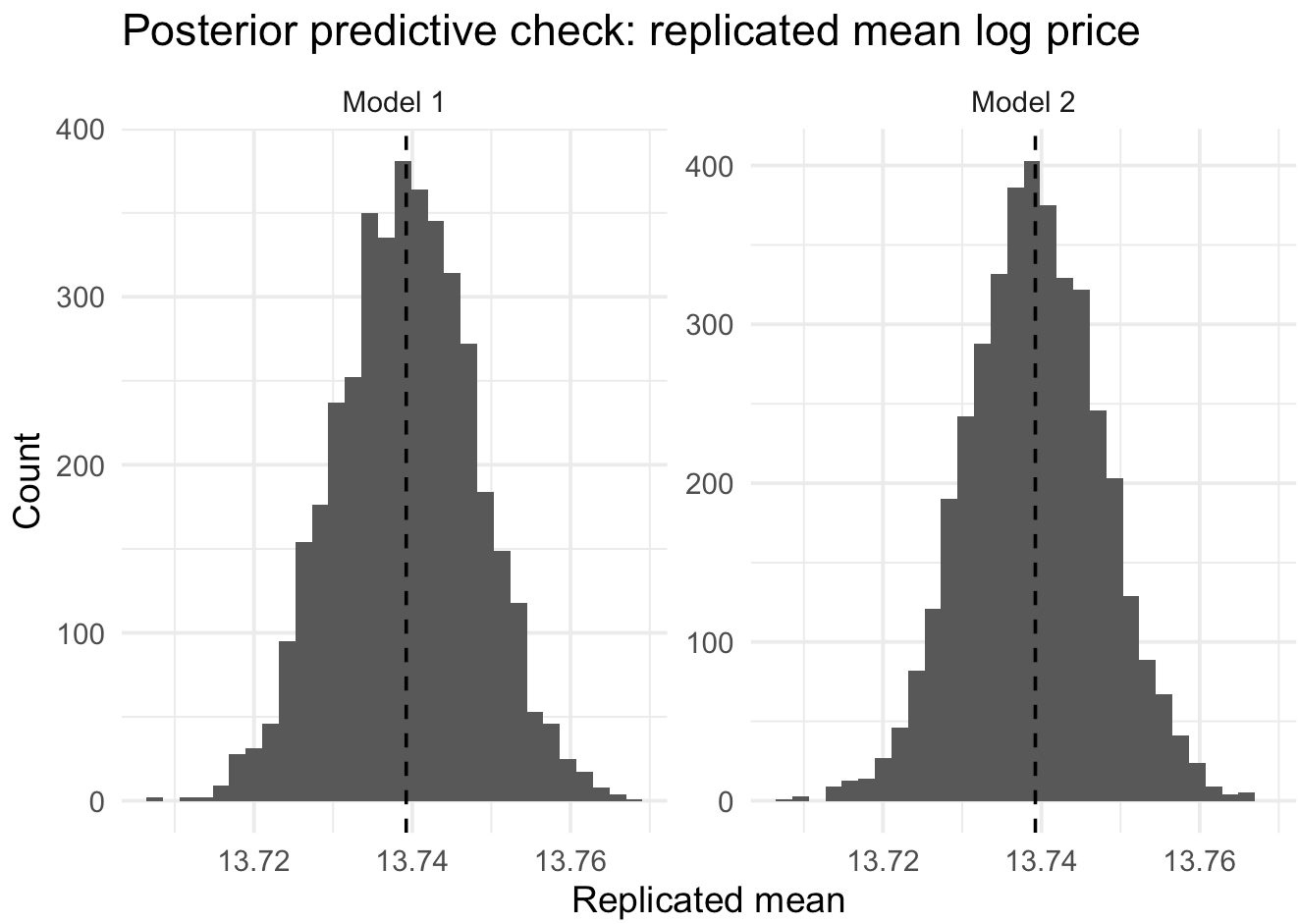
Note: All of the following is a posterior predictive diagnostics

Extract posterior draws

```
yrep_m1 <- fit_m1$draws("y_rep", format = "matrix")
yrep_m2 <- fit_m2$draws("y_rep", format = "matrix")

y_obs <- stan_data_model1$y
```

Diagnostics - PPC

```
ppc_df <- dplyr::bind_rows(
  data.frame(model = "Model 1", y_rep_mean = rowMeans(yrep_m1)),
  data.frame(model = "Model 2", y_rep_mean = rowMeans(yrep_m2))
)

obs_mean <- mean(y_obs)

ggplot(ppc_df, aes(x = y_rep_mean)) +
  geom_histogram(bins = 30) +
  geom_vline(xintercept = obs_mean, linetype = 2) +
  facet_wrap(~ model, scales = "free_y") +
  labs(title = "Posterior predictive check: replicated mean log price",
       x = "Replicated mean", y = "Count") +
  theme_minimal(base_size = 14)
```

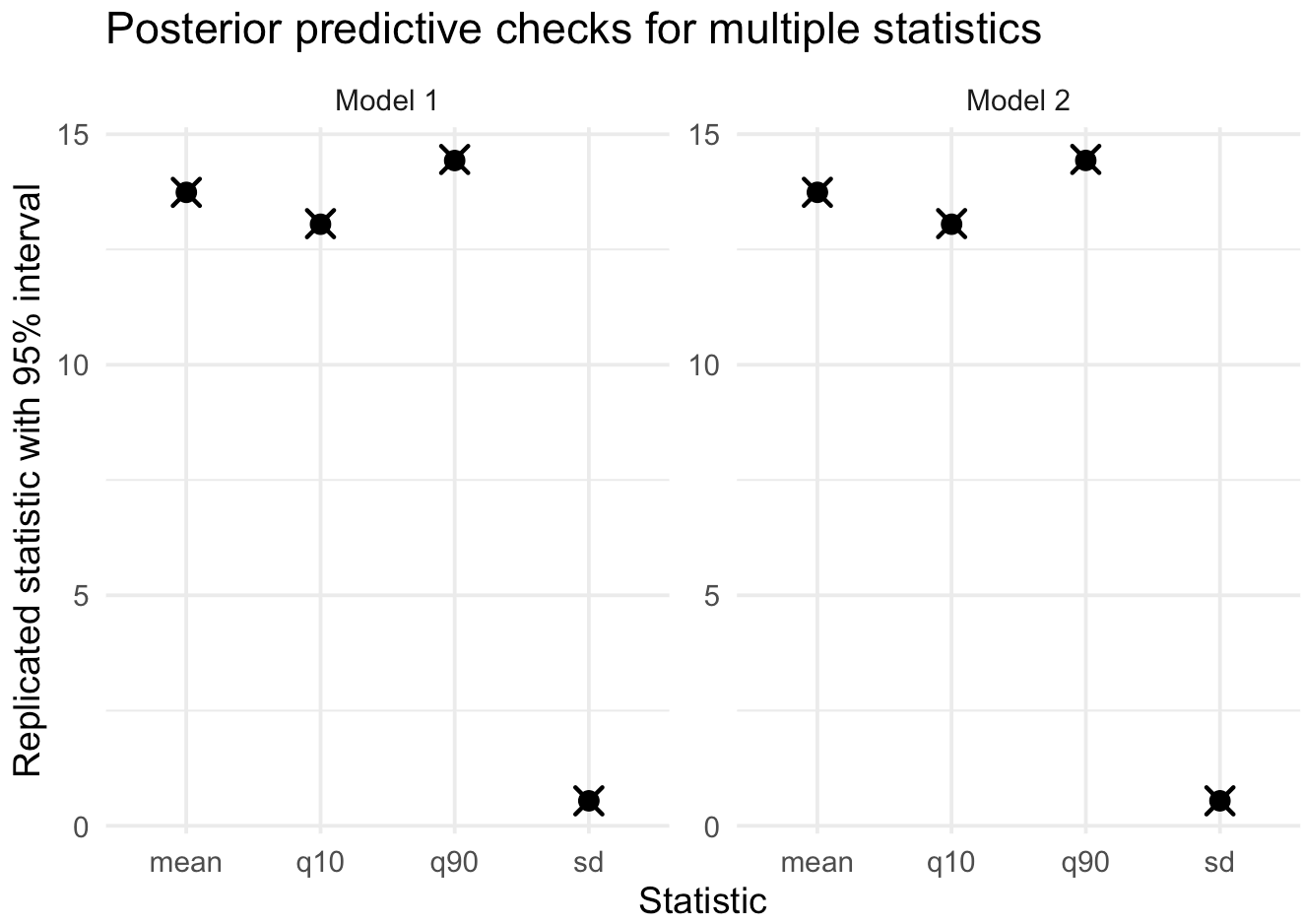


Summary statistics for PCC

```
ppc_stats <- function(yrep, y_obs, model_name) {
  data.frame(
    model = model_name,
    stat = c("mean", "sd", "q10", "q90"),
    observed = c(
      mean(y_obs),
      sd(y_obs),
      quantile(y_obs, 0.10),
      quantile(y_obs, 0.90)
    ),
    rep_mean = c(
      mean(rowMeans(yrep)),
      mean(apply(yrep, 1, sd)),
      mean(apply(yrep, 1, quantile, probs = 0.10)),
      mean(apply(yrep, 1, quantile, probs = 0.90))
    ),
    lower = c(
      quantile(rowMeans(yrep), 0.025),
      quantile(apply(yrep, 1, sd), 0.025),
      quantile(apply(yrep, 1, quantile, probs = 0.10), 0.025),
      quantile(apply(yrep, 1, quantile, probs = 0.90), 0.025)
    ),
    upper = c(
      quantile(rowMeans(yrep), 0.975),
      quantile(apply(yrep, 1, sd), 0.975),
      quantile(apply(yrep, 1, quantile, probs = 0.10), 0.975),
      quantile(apply(yrep, 1, quantile, probs = 0.90), 0.975)
    )
  )
}

ppc_stats_all <- dplyr::bind_rows(
  ppc_stats(yrep_m1, y_obs, "Model 1"),
  ppc_stats(yrep_m2, y_obs, "Model 2")
)

ppc_stats_all
```

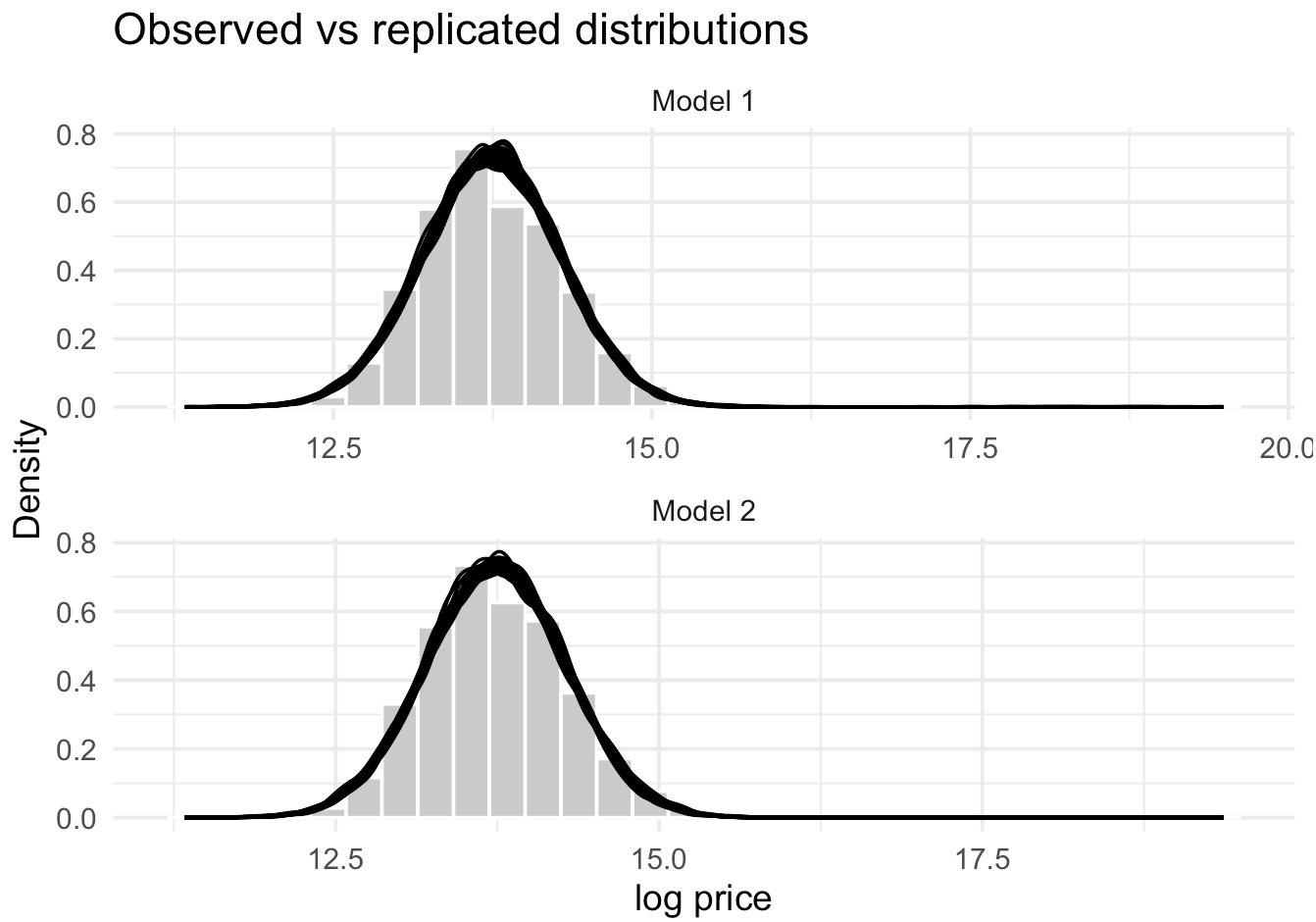
```
##     model stat   observed   rep_mean      lower      upper
## 1 Model 1 mean 13.7392423 13.7392250 13.7222457 13.7565747
## 2 Model 1   sd  0.5418189  0.5420111  0.5298972  0.5547856
## 3 Model 1  q10 13.0604880 13.0475141 13.0205295 13.0741994
## 4 Model 1  q90 14.4521403 14.4294607 14.4029939 14.4562921
## 5 Model 2 mean 13.7392423 13.7390984 13.7223880 13.7559731
## 6 Model 2   sd  0.5418189  0.5419491  0.5296566  0.5539559
## 7 Model 2  q10 13.0604880 13.0467877 13.0202654 13.0723253
## 8 Model 2  q90 14.4521403 14.4302908 14.4038135 14.4551874
```

```
ggplot(ppc_stats_all, aes(x = stat, y = rep_mean)) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.15) +
  geom_point(aes(y = observed), shape = 4, size = 4, stroke = 1.2) +
  facet_wrap(~ model, scales = "free_y") +
  labs(title = "Posterior predictive checks for multiple statistics",
       x = "Statistic", y = "Replicated statistic with 95% interval") +
  theme_minimal(base_size = 14)
```



Observed vs replicated distribution

```
set.seed(123)

draw_ids_m1 <- sample(1:nrow(yrep_m1), 30)
draw_ids_m2 <- sample(1:nrow(yrep_m2), 30)

rep_df_m1 <- data.frame(
  value = as.vector(t(yrep_m1[draw_ids_m1, ])),
  draw = rep(1:30, each = ncol(yrep_m1)),
  model = "Model 1"
)

rep_df_m2 <- data.frame(
  value = as.vector(t(yrep_m2[draw_ids_m2, ])),
  draw = rep(1:30, each = ncol(yrep_m2)),
  model = "Model 2"
)

rep_df_all <- dplyr::bind_rows(rep_df_m1, rep_df_m2)
obs_df <- data.frame(value = y_obs)

ggplot() +
  geom_histogram(data = obs_df, aes(x = value, y = after_stat(density)),
                 bins = 30, fill = "grey80", color = "white") +
  geom_density(data = rep_df_all, aes(x = value, group = interaction(model, draw)),
               alpha = 0.12) +
  facet_wrap(~ model, scales = "free", ncol = 1) +
  labs(title = "Observed vs replicated distributions",
       x = "log price", y = "Density") +
  theme_minimal(base_size = 14)
```



Predictive adequacy check

LOO comparison

```
loglik_m1 <- fit_m1$draws("log_lik", format = "matrix")
loglik_m2 <- fit_m2$draws("log_lik", format = "matrix")

loo_m1 <- loo::loo(loglik_m1)
```

```
## Warning: Some Pareto k diagnostic values are too high. See help('pareto-k-diagnosti
c') for details.
```

```
loo_m2 <- loo::loo(loglik_m2)
```

```
## Warning: Some Pareto k diagnostic values are too high. See help('pareto-k-diagnosti
c') for details.
```

```
print(loo_m1)
```

```
##
## Computed from 4000 by 7130 log-likelihood matrix.
##
##          Estimate    SE
## elpd_loo  -5630.1  80.9
## p_loo        43.2  40.5
## looic     11260.2 161.9
## ------
## MCSE of elpd_loo is NA.
## MCSE and ESS estimates assume independent draws (r_eff=1).
##
## Pareto k diagnostic values:
##                          Count Pct.    Min. ESS
## (-Inf, 0.7]   (good)     7129  100.0%  3860
##    (0.7, 1]   (bad)         0    0.0%  <NA>
##    (1, Inf)   (very bad)    1    0.0%  <NA>
## See help('pareto-k-diagnostic') for details.
```

```
print(loo_m2)
```

```
##
## Computed from 4000 by 7130 log-likelihood matrix.
##
##          Estimate    SE
## elpd_loo  -5123.9  81.3
## p_loo        43.3  36.5
## looic     10247.7 162.5
## ------
## MCSE of elpd_loo is NA.
## MCSE and ESS estimates assume independent draws (r_eff=1).
##
## Pareto k diagnostic values:
##                          Count Pct.    Min. ESS
## (-Inf, 0.7]   (good)     7129  100.0%  3544
##    (0.7, 1]   (bad)         0    0.0%  <NA>
##    (1, Inf)   (very bad)    1    0.0%  <NA>
## See help('pareto-k-diagnostic') for details.
```

```
print(loo_compare(loo_m1, loo_m2))
```

```
##        elpd_diff se_diff
## model2    0.0       0.0
## model1 -506.2      31.5
```